Knowledge Graph Completion with R-GCN and CompGCN

Charan Ponaganti (240150024)
M Charan (240150018) Lalit Deshmane (240150044)

Abstract

Knowledge graphs store facts as relationships between entities, but many useful relationships are missing. This project investigates whether information from neighbouring entities improves missing-link prediction when training must fit on a single GPU. We combine a PySpark preparation pipeline with R-GCN and three CompGCN variants, using DistMult as their scoring function. The experiments use the biomedical ogbl-biokg dataset and compare these graph models with shallow DistMult, TransE and ComplEx models. In the saved run, CompGCN with multiplication achieves the highest MRR: 0.5256 in the local sampled-candidate test and 0.5127 with the OGB evaluator. R-GCN obtains 0.4552 and 0.4013, respectively. These results favour neighbourhood information under the configured training schedule, although they represent one seed rather than a statistically established model ranking. The report explains the data pipeline, model implementation, evaluation protocols and recorded results. It also identifies limitations in the degree-based diagnostic and Spark timing experiment so that their values are not confused with standard MRR or end-to-end processing time.

1 Introduction

A knowledge graph represents a fact as a triple consisting of a head entity, a relation and a tail entity. For example, an illustrative biomedical triple might connect a drug to a protein through an interaction relation. The graph is useful because each entity can participate in several kinds of relationships. It is also incomplete: an absent edge may indicate a relationship that has not yet been recorded, rather than a relationship that is false.

Knowledge graph completion addresses this gap by ranking possible answers to a missing-link query. In tail prediction, the head and relation are known and the model ranks possible tails. Head prediction reverses this task. A useful model should place the recorded answer near the top of the list, while recognising that a query may have more than one valid answer. This is why the candidate set and treatment of other known facts matter as much as the scoring model.

The project asks a practical question: does adding graph neighbourhood information improve link prediction within a modest training budget? A shallow embedding model learns a vector for each entity and relation from observed triples. A graph neural network also updates an entity representation using its neighbours. That extra information may help, but it increases processing cost and makes the implementation more sensitive to graph size and highly connected entities.

Our implementation separates data preparation from model training. PySpark cleans and profiles the triples, prepares identifiers and exports the graph. PyTorch then trains the models on sampled neighbourhoods. This separation makes the Big Data component explicit while keeping the learning stage within the memory available on a Kaggle Tesla T4. An RTX 4050 configuration is included for smaller runs, but the reported measurements come from the T4 configuration.

The main contribution is an implemented comparison with traceable results, rather than a new graph architecture. We compare R-GCN, CompGCN subtraction, CompGCN multiplication and CompGCN circular correlation with three shallow models. We explain both evaluation procedures, examine the trade-off between ranking

quality and training time, and analyse a small R-GCN parameter sweep. We also document where the saved evidence does not support a stronger conclusion.

1.1 Scope of the investigation

The experiments are transductive: the model learns embeddings for the entities represented in the prepared graph. The report does not claim that the trained model can embed an entirely new biomedical entity without further training. It also does not test whether predicted links are clinically correct. The task is benchmark link ranking, evaluated against the dataset's held-out facts.

The results are taken from the repository's results.json and checked against main_results.csv, ogb_results.csv, ablation.json and spark_scale.csv. The self-contained notebook is the implementation reference because it does not import the Python modules under src. Those modules are useful for reading individual concepts, but some details differ from the notebook. The distinction is important when reproducing an experiment: running an illustrative helper is not equivalent to running the saved notebook configuration.

2 Background and Related Work

2.1 Embedding models

DistMult assigns each entity and relation a vector and scores a triple by multiplying their corresponding coordinates and summing the products (Yang et al., 2015). Its simplicity makes it a useful baseline. If graph models cannot improve on it under the chosen conditions, their extra neighbourhood processing may not be justified. Its basic score is symmetric in the two entity vectors, which limits how the shallow model represents asymmetric relationships.

TransE takes a different approach: it treats a relation as a translation from the head vector towards the tail vector (Bordes et al., 2013). A small distance between the translated head and the tail gives a high score. ComplEx uses complex-valued embeddings to represent a richer set of relationships (Trouillon et al., 2016). In this notebook, the real and imaginary parts occupy two halves of the same 128-coordinate storage vector. Thus the comparison uses 64 complex coordinates, not 128 complex coordinates.

These models provide useful reference points because they do not require message passing. In the project, TransE and ComplEx are independently trained shallow models. They are not experiments that replace the decoder while keeping a trained R-GCN or CompGCN encoder fixed. Calling them decoder-only baselines avoids attributing their results to graph representations they never use.

2.2 Learning from relations and neighbours

R-GCN extends graph convolution to graphs with labelled relations (Schlichtkrull et al., 2018). Neighbours connected through different relations contribute through different transformations. Basis decomposition expresses these transformations using a smaller collection of shared matrices. The resulting model can use relation-specific information without learning a completely independent dense matrix for every relation in every layer.

CompGCN combines an entity vector with a relation vector before aggregating messages, and updates relation representations across layers (Vashishth et al., 2020). The composition operator determines how those vectors interact. This project implements subtraction, elementwise multiplication and circular correlation. The implementation is an adaptation for sampled query-side scoring; its results should not be described as a full reproduction of every architectural and experimental choice in the original paper.

Neighbour sampling limits how much of the graph is processed for a mini-batch. The general idea is to select a bounded number of neighbours instead of expanding every adjacent edge, as in sampling-based graph representation learning (Hamilton et al., 2017). This sacrifices some neighbourhood information but makes memory use more manageable. In this project, it is especially relevant because a small number of entities have much higher degree than the median entity.

2.3 Benchmark and processing framework

Open Graph Benchmark provides datasets, splits and evaluation utilities intended to make graph-learning experiments easier to compare (Hu et al., 2020). We use its biomedical knowledge graph and report the OGB evaluator output separately from the notebook's smaller local test. The distinction prevents a sampled score obtained with one candidate construction from being treated as interchangeable with another.

Spark provides DataFrame operations for cleaning, grouping and joining the triples (Zaharia et al., 2016). Here it runs in local mode on one machine. This is a real Spark workflow, but it is not evidence of performance on a multi-machine cluster. PyTorch Geometric supplies the R-GCN convolution used by the notebook (Fey and Lenssen, 2019), while the CompGCN layer and neighbourhood sampler are implemented directly in PyTorch.

3 Dataset and Problem Formulation

3.1 The graph used in the experiments

The recorded experiment uses ogbl-biokg. After cleaning, the saved graph statistics contain 93,773 entities, 51 original relations and 4,762,676 training triples. The original training count in the notebook is 4,762,678, so cleaning removes two rows in total. The export combines duplicates and self-loops in one removal count; it does not identify how many belonged to each category. Table 1 summarises the prepared graph.

Statistic	Value
Entities	93,773
Original relations	51
Training triples	4,762,676
Rows removed in cleaning	2
Message relation identifiers	102
Minimum / median degree	1 / 22
Mean degree	101.58
99th percentile degree	1,279.28
Maximum degree	62,733
Many-to-many relations	51

Table 1: Prepared BioKG training graph. Counts are from the saved Spark statistics.

Entity identifiers require care because biomedical entity types have separate local identifier spaces. The loader applies type offsets to obtain global identifiers, and the Spark stage preserves those numeric identifiers. Without this step, the same local integer could refer to different entities in different types. More subtly, assigning a new identifier order after Spark processing would invalidate the official OGB negative lists unless they were remapped consistently.

The graph has a strongly uneven degree distribution. Its median degree is 22, whereas its mean is about 101.58 and its maximum is 62,733. Degree here counts appearances as a head or tail in the training triples. It is not the number of distinct neighbouring entities: several relations or triples can contribute to the count. This definition explains what the saved summary and degree-based analysis actually measure.

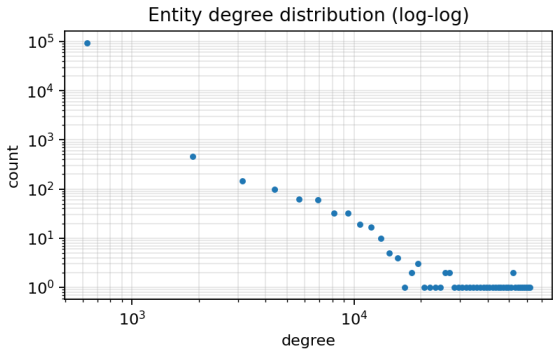


Figure 1: Training-graph degree distribution from the repository. Degrees count head and tail occurrences. The large difference between median and maximum motivates bounded neighbourhood sampling.

The notebook also classifies relations by the average number of tails per head and heads per tail, using a threshold of 1.5. All 51 relations fall into the many-to-many category under this rule. That finding describes the averages for this dataset. It does not imply that every entity pair participates in many relationships, and it leaves no separate one-to-one or one-to-many groups for a category comparison.

3.2 Dataset alternatives and memory

The repository includes loaders for an FB15k-237 smoke test and a degree-filtered Wikidata5M subset. These paths are useful for debugging and exploring larger raw graphs, but neither is the source of the reported result tables. The presence of a loader is not evidence that a complete experiment on that dataset has been performed. The report therefore keeps its empirical conclusions limited to ogbl-biokg.

Memory use involves more than storing the entity vectors. At dimension 128, the BioKG entity table contains 12,002,944 float32 values, or about 48.0 MB in decimal units. Gradients and Adam states add storage, and neighbourhood messages and intermediate activations can consume much more during training. Inverse edges also enlarge the message graph. These costs explain the use of sampling without requiring an unmeasured claim that every possible full-batch implementation must fail.

For larger graphs such as Wikidata5M, even the entity table becomes much larger, and edge processing adds further pressure. The notebook's subset path filters by degree and repeatedly removes low-degree entities. Such a subset changes the learning problem and favours more connected entities, so results on it would need to be reported as subset results. No full Wikidata5M or ogbl-wikikg2 training score is claimed here.

3.3 What constitutes a prediction

Let a triple be written as (h, r, t), where h and t are entities and r is a relation. A scoring function assigns a real number to each possible triple. For a tail query, candidates are compared through scores for (h, r, candidate). For a head query, scores are computed for (candidate, r, t). Larger scores indicate a more plausible answer according to the model.

A ranking is meaningful only relative to its candidate list. Ranking the correct entity among a few hundred sampled alternatives is a different task from ranking it against every entity. The meaning also changes if alternatives are restricted to the correct biomedical type. We therefore state the number of sampled alternatives, the type restriction, the filtering rule and the evaluation direction whenever interpreting a metric.

4 Data Preparation with PySpark

4.1 Ingestion and cleaning

The notebook first loads triples into host memory, constructs pandas DataFrames and passes them to Spark with an explicit schema. If that conversion fails, it writes a tab-separated intermediate file and reads it through Spark. This design makes the ingestion path straightforward on Kaggle, but host memory remains a constraint. It is not a fully distributed reader that streams an arbitrarily large dataset directly from external storage.

Spark removes null values, empty identifiers, duplicate triples and self-loops. It processes the training, validation and test splits separately. Keeping the split distinction is essential: held-out triples must not enter the graph used for learning neighbourhood representations. Removing self-loops is a preprocessing decision, not a universal requirement of knowledge graphs; in another application, a self-relation might carry meaningful information.

The cleaned training DataFrame is cached because several later operations reuse it. Spark computes entity occurrence counts and relation categories. The notebook then collects the degree column into pandas to calculate the median, mean, percentile and extremes. Thus Spark performs the graph aggregation, while the final numerical summary is calculated locally. This is a more accurate description than saying that every reported statistic is computed entirely inside Spark.

4.2 Indexing and graph construction

For the OGB data, numeric global identifiers are preserved. For string-based datasets, the notebook constructs dense mappings and joins them back to the triples. The mappings are derived from the training graph, so validation and test entities not represented there can be excluded by the joins. A future dataset run should record the counts before and after this step to make its evaluation coverage explicit.

An inverse message edge is added for each training triple. If the original edge is (h, r, t), the inverse is (t, r + 51, h) for this run. This produces 102 message-passing relation identifiers. Inverse edges let information travel in both directions, but they do not add new original relation labels to the prediction task. The scoring relation table still corresponds to the 51 original relations.

The pipeline writes indexed splits, entity and relation mappings, inverse-edge training data, relation categories and known-answer tables as Parquet files. Parquet provides a structured handoff that can be inspected independently of training. The exported tables also make it easier to determine whether a later problem arose in identifier mapping, graph construction or model scoring.

4.3 Filtering and the handoff to PyTorch

Filtered evaluation avoids penalising a model for ranking another known correct answer above the selected target. The notebook builds known-triple information from the union of training, validation and test triples. This union is used to mask alternatives during evaluation, not to construct the training message graph. Using held-out facts solely for filtering is distinct from using them as training edges.

The GPU lookup encodes an indexed triple as key = (h × R + r) × E + t, where E and R are the entity and original-relation counts. Sorted 64-bit

keys allow vectorised membership checks. For this graph, even the largest possible key is below 4.49 × 10^11, comfortably inside the signed 64-bit range. Correct identifier bounds are necessary for this encoding to avoid collisions.

Finally, the notebook converts the prepared data into arrays and tensors, constructs the adjacency used by the sampler, and stops Spark before GPU training. Stopping the session releases resources associated with the Spark process and reduces competition for host memory. The overall flow is therefore ingestion, cleaning, indexing, export, graph sampling, training and evaluation, with saved files connecting the stages.

5 Model Design and Training

5.1 Shared scoring and the shallow baseline

DistMult uses the score s(h, r, t) = Σj hj rj tj, summed over the embedding coordinates. For the shallow baseline, all three vectors are taken directly from learned embedding tables. For the graph models, the entity on the known query side is contextualised by the encoder. A residual connection adds its original embedding back to the encoded vector before scoring candidates.

This query-side design avoids encoding every candidate neighbourhood for every query. Candidates use the global entity embedding table. In tail prediction, the known head is encoded; in head prediction, the known tail is encoded. The resulting system is less expensive than contextualising both sides for every candidate, but it does not have the same behaviour as a fully encoded graph scorer. This implementation choice is part of the method being evaluated.

5.2 R-GCN with shared bases

Each R-GCN layer combines transformed messages from neighbouring entities and a transformation of the entity itself. A relation determines which message transformation is used. Neighbourhood normalisation prevents a simple sum from growing only because more messages arrive. The notebook uses PyTorch Geometric's RGCNConv and applies ReLU and dropout between layers.

With basis decomposition, a relation matrix is expressed as Wr = Σb arb Vb. The Vb matrices are shared, and the arb coefficients select their mixture for relation r. The recorded configuration has two layers, eight bases and 128-dimensional embeddings. Increasing the number of bases offers more relation-specific flexibility, while fewer bases reduce the number of matrix parameters. The ablation experiment later checks whether that additional flexibility helps within the shorter schedule.

The message graph includes both original and inverse relation identifiers. R-GCN can therefore use separate transformations for these directions. Its decoder relation embeddings are learned separately from the convolution matrices. This separation is useful to understand: the convolution determines how neighbourhood evidence changes an entity vector, and the decoder determines how that vector is scored for a query relation.

5.3 CompGCN implementation

CompGCN first composes the source entity vector with a relation vector. Subtraction computes e − r, multiplication computes e ⊙ r, and circular correlation combines coordinates through a circular shift interaction implemented with real FFT operations. The composed message is transformed by an incoming or outgoing weight matrix. The layer averages received messages and adds a separate linear transformation of the entity itself.

In this implementation, inverse edge identifiers map back to the same underlying relation embedding as their original relation. Direction is distinguished by the incoming and outgoing matrices. A learned relation transformation updates the relation table after each layer, and the final transformed relations are used by the decoder. These details distinguish the implementation from a generic equation that leaves inverse relations and self-loops unspecified.

The multiplication variant starts its relation vectors at ones, so the initial composition preserves the entity vector. Subtraction and correlation use Xavier initialisation for the relation table. Consequently, the comparison changes both the operator and its associated initialisation. The results establish how the implemented variants behave; they do not isolate the operator as the only causal difference.

All composition operators run in float32 with autocast disabled inside the composition step. Other CUDA operations can still use mixed precision. This protects the FFT path from half-precision constraints, but it does not guarantee that correlation will optimise well. A poor score after

this precaution is evidence about the saved configuration, not proof that circular correlation is unsuitable for knowledge graphs.

5.4 Sampling and optimisation

The sampler stores adjacency in compressed sparse row form and samples bounded neighbourhoods around the query entities. The configured fanout is ten neighbours at each of two expansion steps. Sampling is with replacement, so repeated neighbours are possible and the fanout is not a promise of ten distinct neighbours. The sampled edges are assembled into a local graph for message passing.

Each optimisation step includes both head and tail prediction. The training batch contains 512 triples, with 16 negative alternatives for each direction. On BioKG, negative sampling uses the entity type of the argument being corrupted. Harder negatives receive greater weight through a detached softmax of their scores, following the self-adversarial weighting idea used by RotatE (Sun et al., 2019).

The loss averages the head and tail objectives. Adam uses a learning rate of 0.001 and weight decay of 0.00001, with gradient clipping at 1.0. Dropout is 0.2 between graph layers. Although a label-smoothing constant appears in the configuration, the active self-adversarial loss does not use it; smoothing applies only to the alternative binary cross-entropy path. It should therefore not be listed as an active feature of this run.

Each configured epoch samples 400 batches with replacement rather than making a complete pass through the training graph. Eight such epochs allow at most 3,200 updates and 1,638,400 sampled triple presentations per model before early stopping or skipped batches. This is fewer presentations than the number of training triples, and repeated samples reduce distinct coverage further. The word epoch in this notebook therefore denotes a fixed training interval, not full-dataset coverage.

5.5 Checkpoints and practical safeguards

Validation selects the best checkpoint using tail-only sampled MRR. The current notebook uses up to 800 validation triples and 100 sampled negatives per query, with patience five. This validation procedure is smaller than the final test and does not use the official OGB score for checkpoint selection. Selecting on tail performance may also matter when head and tail results differ substantially.

The training loop skips non-finite losses and attempts to reduce batch size and fanout after a GPU out-of-memory error. It also saves partial results and model checkpoints so an interrupted session can resume. These safeguards improve usability, but successful-step counts and any changed sampling settings should be recorded when comparing runs. Equal configured schedules do not necessarily imply equal numbers of completed updates.

6 Experimental Setup and Metrics

6.1 Recorded configuration

The saved run identifies a Tesla T4 with approximately 15.6 GB of memory, the fast budget and QUICK_RUN set to false. Seed 42 is recorded for evaluation and used by the notebook's seeding routine. Table 2 lists the main settings. The RTX 4050 preset uses smaller dimensions, batches and fanout; it is a supported configuration rather than a second measured experiment in this report.

Setting	Value
Embedding coordinates	128
GNN layers / R-GCN bases	2 / 8
Batch size	512
Fanout per hop	10, 10
Maximum epochs × steps	8 × 400
Negatives per direction	16
Learning rate	0.001
Weight decay	0.00001
Dropout / gradient clip	0.2 / 1.0
Local test triples	8,000
Local sampled negatives	500
OGB test triples	162,870

Table 2: Main-run settings from the saved configuration. Epochs are fixed sampled intervals.

All seven models have the same configured maximum epochs, steps per epoch and number of training negatives. This is a matched update schedule, not matched computation. One R-GCN update costs more than a DistMult update, and early stopping can shorten a run. Likewise, the models do not have identical parameter counts or exactly identical random samples, because their execution consumes randomness differently.

6.2 Interpreting ranking metrics

For a query with rank q, reciprocal rank is 1/q. Mean reciprocal rank, or MRR, averages these values over all evaluated queries. Mean rank, or MR, averages q itself. Hits@k is the fraction of queries for which q is at most k. Higher MRR and Hits@k are better; lower MR is better. MRR emphasises getting answers very near the top, whereas MR can be strongly affected by a few very poor ranks.

For example, suppose four queries have ranks 1, 2, 5 and 10. Their MR is 4.5 and their MRR is (1 + 0.5 + 0.2 + 0.1)/4 = 0.45. Hits@1 is 0.25, Hits@3 is 0.50 and Hits@10 is 1.00. This example is illustrative, not an additional experiment. It shows why a score of 0.45 is not the same as saying that 45 percent of predictions are correct at rank one.

6.3 Local sampled-candidate evaluation

The local test takes the first 8,000 triples of the prepared test array. Each triple generates a head query and a tail query, giving 16,000 directional ranks. A query contains the true entity plus 500 sampled alternatives. The saved n_candidates value describes the alternatives, so the initial score matrix has 501 entries per query. Sampling with replacement can repeat alternatives, meaning these are not necessarily 501 distinct entities.

The candidate generator is the same type-aware corruption helper used in training. With the saved BioKG settings, the local test is therefore also type-aware; it is not an unconstrained sample from all entities. Other known true triples are masked through the lookup table. A tie with the positive score counts against the target because the ranking code uses greater-than-or-equal comparisons for negatives. This is a pessimistic tie rule.

The candidate generator is reset to seed 42 for each evaluation, which aligns candidate sampling between models under the same batching. Graph neighbourhood sampling uses the global random generator separately, so contextualised queries can still vary between evaluation calls. The first 8,000 prepared rows are also not documented as a stratified test sample. Their results should be read as a local diagnostic, not a complete replacement for the benchmark test.

6.4 OGB evaluation

The second procedure scores all 162,870 original OGB test triples in both directions and passes the positive and supplied negative scores to the OGB evaluator. The notebook preserves the original test order so that each positive remains paired with its corresponding negative list. The saved metrics average the evaluator's per-query outputs across head and tail predictions.

OGB's BioKG protocol supplies 500 type-compatible negative entities for each corruption direction (Open Graph Benchmark, n.d.). Both procedures therefore involve sampled negatives, but they differ in which triples are evaluated, which negative lists are used, filtering and tie handling. The OGB results are the more appropriate benchmark reference. A difference between the two columns cannot be explained simply as unrestricted candidates versus type-restricted candidates.

7 Results and Discussion

7.1 Local ranking results

Table 3 reports the saved local MRR and Hits metrics, rounded to four decimal places. CompGCN-mult leads with MRR 0.5256 and Hits@10 0.8005. CompGCN-sub follows at 0.5000 MRR, then R-GCN at 0.4552. Among the shallow models, ComplEx obtains 0.3866, ahead of DistMult and TransE at approximately 0.314. Correlation is much weaker in this run, with MRR 0.0725.

Model	MRR	H@1	H@3	H@10
DistMult	0.3141	0.2136	0.3671	0.5228
R-GCN	0.4552	0.3189	0.5349	0.7290
CompGCN-sub	0.5000	0.3545	0.5816	0.7944
CompGCN-mult	0.5256	0.3859	0.6084	0.8005
CompGCN-corr	0.0725	0.0441	0.0750	0.1152
TransE	0.3137	0.1786	0.3632	0.6165
ComplEx	0.3866	0.2740	0.4493	0.6099

Table 3: Local filtered sampled-candidate results on 8,000 triples, evaluated in both directions.

CompGCN-mult improves on R-GCN by 0.0704 MRR in the local test, and on DistMult by 0.2115. These are absolute differences on the MRR scale. They describe the saved comparison and are not estimates of a guaranteed gain on other datasets or seeds. The smaller gap between the multiplication and subtraction variants is particularly worth checking with repeated runs before choosing between them solely on accuracy.

MR tells a slightly different story. CompGCN-sub has the lowest mean rank, 14.59, compared with 15.80 for multiplication and 26.90 for R-

GCN. Multiplication can therefore win on MRR while subtraction wins on MR: a model may produce more top-ranked answers yet still have some relatively poor cases. Reporting several metrics makes this trade-off visible rather than hiding it behind a single overall winner.

7.2 OGB results

Table 4 gives the OGB evaluator results. CompGCN-mult again has the highest MRR, 0.5127, followed by CompGCN-sub at 0.4995 and R-GCN at 0.4013. The top three models retain their order across the two procedures. Their agreement supports the main observation that the implemented graph encoders outperform the shallow DistMult baseline within this schedule.

Model	MRR	H@1	H@3	H@10
DistMult	0.1492	0.0964	0.1692	0.2507
R-GCN	0.4013	0.2900	0.4507	0.6224
CompGCN-sub	0.4995	0.3848	0.5558	0.7255
CompGCN-mult	0.5127	0.3984	0.5719	0.7355
CompGCN-corr	0.0483	0.0275	0.0459	0.0754
TransE	0.2651	0.1600	0.2994	0.4861
ComplEx	0.2603	0.1761	0.2879	0.4240

Table 4: OGB evaluator results on all 162,870 original test triples, with head and tail queries.

The full ordering is not identical. TransE has OGB MRR 0.2651, slightly above ComplEx at 0.2603, although ComplEx leads it in the local test. DistMult falls from 0.3141 locally to 0.1492 with OGB, while multiplication changes from 0.5256 to 0.5127. These differences show that the protocol and test population affect models differently. They should not be used to infer a single cause without controlling those differences experimentally.

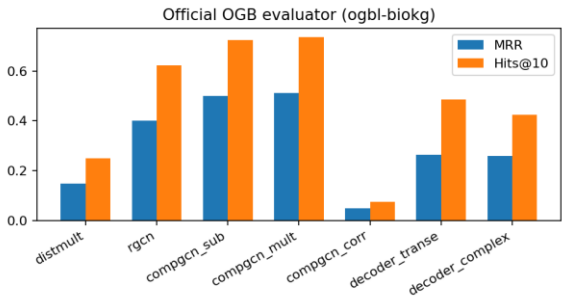


Figure 2: Saved OGB MRR and Hits@10 results. Multiplication has the strongest recorded performance, with subtraction close behind. These are single-run scores without confidence intervals.

The absolute OGB MRR advantage of multiplication over R-GCN is 0.1114; its advantage over subtraction is 0.0132. The latter is a much narrower margin. No standard deviations or confidence intervals are available, so the report does not call either difference statistically significant. The evidence supports identifying the strongest recorded configuration, rather than claiming a universal superiority of CompGCN multiplication.

7.3 Head prediction and tail prediction

The saved local metrics separate the two directions. Multiplication reaches tail MRR 0.6565 and head MRR 0.3947. R-GCN shows a similar imbalance, at 0.5723 and 0.3382. DistMult has a smaller difference, 0.3325 versus 0.2956, but lower performance in both directions. Table 5 makes the asymmetry explicit.

Model	Tail	Head	Gap
DistMult	0.3325	0.2956	0.0369
R-GCN	0.5723	0.3382	0.2341
CompGCN-sub	0.6142	0.3859	0.2284
CompGCN-mult	0.6565	0.3947	0.2619
CompGCN-corr	0.0768	0.0682	0.0086
TransE	0.3671	0.2602	0.1070
ComplEx	0.4501	0.3231	0.1270

Table 5: Directional MRR in the local test. Gap is tail MRR minus head MRR.

For multiplication, the gap is 0.2619. This suggests that improvements should be checked separately for the two directions rather than only through their average. Possible influences include the relation distribution, candidate pools, query-side encoding and tail-only checkpoint selection. The current experiment does not isolate these factors. A useful follow-up would evaluate per-relation head and tail metrics and select checkpoints using a bidirectional validation score.

7.4 Training cost

The exported train_hours values sum timed training epochs and exclude validation and final evaluation. Converting them to minutes gives approximately 0.66 for DistMult, 13.80 for R-GCN, 3.30 for subtraction, 3.36 for multiplication, 3.00 for correlation, 0.72 for TransE and 0.78 for ComplEx. The small times for shallow models explain why equal update counts do not mean equal resource use.

Multiplication's recorded training time is about one quarter of R-GCN's, while its OGB MRR is

higher. That makes it an attractive configuration within this implementation. This is not a benchmark of optimally tuned R-GCN against optimally tuned CompGCN: different kernels, parameterisations and sampling interactions can affect the timing. The result is a practical comparison of the supplied code paths.

The JSON records a notebook wall time of 0.86 hours, or 51.6 minutes. Since the notebook supports resuming previously completed work, this field should not automatically be interpreted as a guaranteed clean-start runtime for reproducing every artifact. A future run should save whether it resumed, dependency versions, successful update counts and peak memory alongside the time measurements.

8 Ablations and Error Analysis

8.1 R-GCN parameter sweep

The R-GCN ablations use four epochs of 200 steps each. This is 800 configured updates, one quarter of the 3,200-update main schedule, even though both the number of epochs and steps per epoch are halved. Comparisons within Table 6 are meaningful as a shorter-budget sweep. They should not be treated as equal-budget alternatives to the main R-GCN result.

Setting	MRR	H@1	H@10
Short-budget base	0.4190	0.2786	0.7104
One layer	0.4319	0.2911	0.7178
Eight negatives	0.4304	0.2922	0.7117
Four bases	0.4368	0.3030	0.7138
Dimension 64	0.4100	0.2648	0.7083

Table 6: R-GCN ablations with 800 configured updates each. The main schedule allows 3,200 updates.

The four-basis setting has the highest MRR in the sweep, 0.4368, compared with 0.4190 for the short-budget base. Reducing depth to one layer gives 0.4319, and reducing negatives to eight gives 0.4304. Reducing dimension to 64 gives 0.4100. The observed changes are small enough that repeated seeds would be useful before treating one setting as reliably better.

The sweep suggests that a larger model is not automatically better under limited training. Fewer bases or a shallower encoder may be easier to optimise within 800 updates, but the current data do not establish that explanation. The one-layer variant also has the highest Hits@10 in this sweep, 0.7178, while four bases gives the best MRR. The preferred setting depends partly on whether the application values the first few ranks or a broader top-ten list.

8.2 What the degree diagnostic measures

The repository contains degree-based values labelled MRR, but the code computes them differently from the main metric. It first averages the head and tail ranks for each triple, then takes the reciprocal. Its per-triple value is therefore 2/(q_head + q_tail). Standard bidirectional MRR instead averages 1/q_head and 1/q_tail. These expressions are generally unequal.

For example, head and tail ranks of 1 and 9 produce standard bidirectional MRR of about 0.5556. Taking the reciprocal of the mean rank gives 0.2. The second statistic penalises an imbalance between directions more strongly. Consequently, the saved category value 0.3542 should not be compared directly with multiplication's main MRR of 0.5256, nor explained away as merely a different slice of the same metric.

Table 7 retains the saved degree values but labels them as a reciprocal-mean-rank diagnostic. The groups are quartiles of head occurrence degree among evaluated triples, not quartiles of all entities in the graph. Common entities can appear in many evaluated triples, so this grouping has a different interpretation from dividing the entity set into four equally sized groups.

Head occurrence degree	Diagnostic
1 to 121	0.1764
Above 121 to 1,583	0.2326
Above 1,583 to 8,071	0.4578
Above 8,071 to 62,733	0.5503

Table 7: Saved reciprocal-mean-rank diagnostic by query-head degree. These values are not standard MRR.

The diagnostic rises from 0.1764 in the lowest-degree group to 0.5503 in the highest. This is consistent with poorer ranking for less connected query heads under the diagnostic, but it does not prove that degree alone causes the difference. Relation composition and entity type may differ between groups. Standard per-group MRR requires the separate directional ranks; those ranks are not present in the exported aggregate JSON, so corrected values cannot be reconstructed from it.

8.3 Correlation and embedding visualisation

Correlation obtains the lowest MRR in both result tables. The saved training-curve figure can help identify when validation performance deteriorated, but aggregate metrics alone do not diagnose the cause. Learning rate, initialisation, relation-vector scale and gradient behaviour are reasonable variables to investigate. A controlled follow-up should change one factor at a time and retain loss, validation and gradient logs.

The repository also includes a two-dimensional projection of base entity embeddings using UMAP or a t-SNE fallback. It is useful as a visual inspection of embedding structure, but the scatter plot has no entity-type colouring or quantitative cluster evaluation. Apparent groups therefore cannot establish that the model recovered biomedical types. That claim would require labels and a separate analysis.

9 Spark Timing and Scalability

9.1 Operations that were timed

The Spark experiment has two parts. The fraction sweep runs a count, an average-degree aggregation and a relation-group count on 10, 25, 50 and 100 percent of the training DataFrame. The partition sweep times repartitioning followed by a relation-group count. These are different workloads; the partition timings do not include the degree calculation used in the fraction sweep.

Setting	Approx. rows	Seconds
10% fraction	476,267	1.94
25% fraction	1,190,669	1.65
50% fraction	2,381,338	2.06
100% fraction	4,762,676	0.73
2 partitions	4,762,676	1.58
4 partitions	4,762,676	1.31
8 partitions	4,762,676	1.58
16 partitions	4,762,676	1.37

Table 8: Single-run local Spark timings. Fraction and partition sweeps time different workloads.

The approximate row counts for sampled fractions are calculated as the full count multiplied by the fraction. They are not saved realised sample counts. The 100-percent DataFrame has already been cached and used earlier, while smaller fractions add sampling work. Spark can also reuse or optimise existing plans. These conditions make the sweep unsuitable for inferring a clean linear relationship between input size and runtime.

9.2 Reading the observed times

The full-data point takes 0.73 seconds, less than the 1.94 seconds for the 10-percent sample. This does not mean that processing more data is inherently faster. It reflects a timing experiment in which cache state and query work differ. Nor does it mean that complete ingestion, cleaning, indexing, Parquet writing and tensor preparation finish in under a second. Those stages are outside the timed operation group.

The partition experiment ranges from about 1.31 to 1.58 seconds, with four partitions the fastest observed setting. There is one measurement per setting and no estimate of normal run-to-run variation. The data do not establish a best partition count or demonstrate saturation of a specific CPU configuration. They show that these small local workloads have similar observed times under the tested partition choices.

9.3 A stronger follow-up experiment

A more informative study would measure the same operation at every setting, record actual row counts and separate cold-cache from warm-cache trials. Several repetitions should be run in a varied order and summarised with medians and spread. End-to-end ETL time, shuffle volume, peak host memory and task counts would help explain where resources are spent.

For cluster scalability, the number of workers would need to vary while the dataset and operations remain controlled. The current local-mode results do not answer that question. They still document that Spark is integrated into the preparation pipeline, but the scope of the scalability claim should remain limited to a small single-machine timing study.

10 Reproducibility and Project Organisation

The main executable artifact is notebooks/kg_completion_rgcn_compgcn.ipynb. It contains configuration, loaders, Spark transformations, model classes, training, evaluation and export cells. The files under src separate several of these ideas into readable modules, but the notebook is self-contained. requirements.txt lists dependencies, while

report/figures holds the exported charts and the report directory holds the written analysis.

To reproduce the intended experiment, first use QUICK_RUN for a small smoke test and verify that real data were loaded. The notebook has a synthetic fallback when data cannot be obtained; such a run is suitable for checking execution, not for reporting benchmark performance. Next choose BioKG, disable QUICK_RUN and use the recorded T4 configuration. Inspect the saved flags rather than relying on hardware defaults from an older README.

For a fresh comparison, use a new output directory or disable resume after retaining previous artifacts. The current resume tag does not encode every training and evaluation option. Reusing an output directory after changing an omitted option can therefore mix incompatible saved results. Keeping a full configuration hash, notebook revision and dataset version with each run would make this safer.

After training, retain checkpoints, histories, results.json, both result CSV files, the ablation export and Spark timings together. The final report tables should be generated from those records rather than copied from a chart by eye. Check that all seven model rows are present, that the official evaluator used original OGB ordering, and that no skipped or failed model has been silently omitted from the comparison.

A full rerun is still necessary to verify the reported numerical results independently. The repository provides aggregate outputs, but not every per-query score and training event needed to reconstruct them without execution. The present analysis checks internal agreement of the saved tables and consistency with the notebook; it does not treat that check as a new training experiment.

11 Conclusion

The project demonstrates a complete workflow for preparing a relational graph with Spark and training sampled graph encoders on one GPU. In the recorded BioKG run, CompGCN multiplication has the strongest ranking performance, with local MRR 0.5256 and OGB MRR 0.5127. Subtraction is close behind under OGB evaluation, while R-GCN improves substantially over the shallow DistMult baseline. Multiplication also has a lower recorded training-loop time than R-GCN in this implementation.

The results support using neighbourhood information within the tested update schedule. They do not establish a universal ordering of architectures, and they do not show that equal steps are equal computation. Head prediction remains weaker than tail prediction. The shorter R-GCN sweep suggests useful lower-cost settings, but repeated seeds are needed before turning those observations into firm recommendations.

The next priorities are to preserve directional ranks for correct group metrics, evaluate several seeds, compare models under both equal-update and equal-time budgets, and repeat Spark timings with controlled cache state. These changes would strengthen the evidence without changing the central aim of the project: understanding when graph structure improves missing-link ranking under practical resource limits.

Limitations

The study covers one dataset and one recorded seed. No uncertainty intervals or repeated-run summaries are available. The local test uses a prefix of the prepared test array, and both test procedures use sampled negatives rather than exhaustive ranking over all entities. Their numerical values should therefore remain attached to their stated protocols.

The encoder contextualises only the known query entity, and neighbour sampling omits part of the surrounding graph. Checkpoint selection uses tail-only validation. The type-aware corruption helper also replaces an accidental true-entity draw with the next global identifier; at an entity-type boundary, that replacement can leave the intended type pool. It does not filter all other true triples during training. These implementation details should be addressed in future evaluation and sampling checks.

The degree diagnostic is not standard MRR, and its corrected values cannot be recovered from aggregate results. Correlation's weak performance is specific to the implemented settings. The Spark measurements do not establish cluster scalability or end-to-end throughput. Peak GPU memory and a complete software-version record are not included in results.json, so no precise memory-efficiency claim is made.

Ethics Statement

The benchmark contains biomedical relationships, but a high link score is not evidence that a drug interaction or biological claim is clinically valid. Predictions would require domain review and independent validation before any practical use. Missing facts and uneven coverage can also bias ranking towards well-connected entities. The project uses benchmark data and reports model behaviour without presenting inferred links as confirmed biomedical discoveries.

References

Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. Advances in Neural Information Processing Systems.

Matthias Fey and Jan E. Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. ICLR Workshop on Representation Learning on Graphs and Manifolds. arXiv:1903.02428.

William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. Advances in Neural Information Processing Systems.

Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for machine learning on graphs. Advances in Neural Information Processing Systems.

Open Graph Benchmark. n.d. Link property prediction documentation, ogbl-biokg. https://ogb.stanford.edu/docs/linkprop/ Accessed 7 November 2026.

Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. The Semantic Web, ESWC. https://arxiv.org/abs/1703.06103

Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. 2019. RotatE: Knowledge graph embedding by relational rotation in complex space. International Conference on Learning Representations.

Theo Trouillon, Johannes Welbl, Sebastian Riedel, Eric Gaussier, and Guillaume Bouchard. 2016. Complex embeddings for simple link prediction. Proceedings of the 33rd International Conference on Machine Learning.

Shikhar Vashishth, Soumya Sanyal, Vikram Nitin, and Partha Talukdar. 2020. Composition-based multi-relational graph convolutional networks. International Conference on Learning Representations. https://arxiv.org/abs/1911.03082

Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. 2015. Embedding entities and relations for learning and inference in knowledge bases. International Conference on Learning Representations.

Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: A unified engine for big data processing. Communications of the ACM, 59(11):56-65.